Quantitative Process Management (QPM)

Doc ID: HH-CMM-03 **Version:** 1.0 **Effective Date:** 2026-08-29 **Owner:** Venkat Narasimha (Processbricks) **Review Cycle:** Annual (next: 2027-08-29) **Last Reviewed:** 2026-08-29

> **Classification:** Internal **One-line summary:** We commit to measuring Haritha's process — uptime, latency, deploy frequency, backup success — and using those measurements, not gut feel, to drive decisions.

1. Purpose

Level 4 in CMM exists for one reason: gut feel is wrong more often than we'd like, and "the system seems fine" can hide "the system has been down 4% of the time for three months". Quantitative Process Management (QPM) replaces gut feel with metrics that have **baselines**, **control limits**, and **deltas we can read**.

Without QPM, every decision is an opinion:

- "Is the system fast enough?" → opinion.
- "Should we deploy today?" → opinion.
- "Did last week's deploy make things better or worse?" → opinion.

With QPM, every decision is a measurement against an expected range:

- "Is the system fast enough?" → p95 login latency vs target.
- "Should we deploy today?" → current error rate vs deploy window.
- "Did last week's deploy make things better?" → change-failure-rate delta.

This document defines the metrics Haritha tracks, the targets, the control limits, and the gaps between today's measurement and tomorrow's. It's the input to PDCA cycles (10-process-improvement) and to the maturity assessment (09-cmm-maturity §3a).

The scope is deliberately small. We don't need 200 metrics to reach Level 4; we need 5-8 well-defined metrics that we actually measure and react to. More metrics, less reaction = noise.

2. Scope

2.1 In scope

- **What QPM is** — origin, statistical-process-control concept, baseline + control limit terminology.
- **Metrics catalogue** — 8 metrics Haritha tracks, each with definition, target, baseline, source.
- **Today's measurement** — what we have, what we don't, what we'd add.
- **Dashboards** — what should be on a single screen.
- **Reporting cadence** — weekly summary, monthly review, annual target reset.
- **Tooling** — Frappe built-in monitoring + custom probes; no Prometheus/Grafana in v1.

2.2 Out of scope

- **Application-level business metrics** (e.g., "number of patient records entered this month") — those belong in Frappe's own reports, not in this process-management doc.
- **Security metrics** (failed logins, blocked IPs) — see 01-info-security §5 + 02-access-control §5.
- **Vendor SLAs** (Frappe community forum, DuckDNS, Let's Encrypt) — see 08-business-continuity §6.

- **CMMI-style QPM full process area** — this doc covers the spirit (data-driven decisions) not the letter (no statistical-process-control sub-practices).

3. Policy Statement

3.1 What we commit to

Haritha Hospitals commits to:

1. **The metrics in §3a.2 are the official metrics.** New metrics may be added via PDCA cycle; existing metrics may be removed only via an annual review.
2. **Every metric has a target and a current value.** A metric without a target is a vanity metric; we don't track those.
3. **Every metric has a source of truth.** A metric we can't measure today is in §3a.3 "gaps" until we can.
4. **Weekly summary + monthly review cadence.** Weekly = heartbeat summary; monthly = Venkat review; annually = target reset.
5. **Control limits are explicit, not implicit.** If we say "target = 99.9% uptime", we also say "alert if uptime drops below 99.5% for 7 days running".
6. **No metric theatre.** A metric that nobody reads is not a metric; it's a log entry. Quarterly audit: who's reading this metric and what decision does it drive? If the answer is "nobody", remove it.

3.2 What QPM is and is not

QPM is:

- A way to replace opinion with data.
- A way to detect drift before it becomes an incident.
- A way to compare "before" and "after" a change objectively.
- A prerequisite for 10-process-improvement — PDCA needs metrics to test hypotheses.

QPM is NOT:

- A replacement for incident response (07-incident-management). Metrics are signals, not actions.
- A way to predict the future. "99.9% uptime target" doesn't mean we'll hit it.
- A surveillance tool. Metrics are aggregated; we don't log every request.
- A substitute for judgement. The metric says "MTTR spiked"; the operator decides whether that's noise or signal.

3.3 The baseline / target / control-limit model

Every metric in §3a.2 follows the same triple:

Concept	Definition	Example
Baseline	What we typically see today. Established by 30+ days of measurement.	Login p95 = 280ms over the past 30 days
Target	What we want to achieve. Set annually.	Login p95 target = 500ms
Control limit	When to alert. Usually baseline ± N standard deviations, OR a hard floor/ceiling.	Alert if p95 > 1000ms for 3 consecutive samples

Why this matters: a target without a baseline is wishful thinking. A control limit without a target is just noise. The triple is what makes a metric useful.

3a. Current State (as of 2026-08-29)

3a.1 The 8 official metrics

#	Metric	Definition	Target	Today's baseline	Source	Control limit (alert when…)
M1	**Availability (per env)**	Uptime % over rolling 30 days. `100% - (down-time_seconds / to-tal_seconds)`.	99.9% prod, 99.0% dev/qa	~99.95% prod (informal), 99.0% dev	Heartbeat probe + post-mortem timeline	< 99.5% prod for any 7-day window, OR any SEV-1
M2	**MTTR (Mean Time To Recover)**	Average time from "incident detected" to "all-clear sent", per SEV level.	SEV-1 ≤ 30 min; SEV-2 ≤ 4 hours	SEV-1: ~10 min (2026-08-29 only); SEV-2: n/a	Post-mortem timelines	SEV-1 > 60 min; SEV-2 > 8 hours
M3	**MTBF (Mean Time Between Failures)**	Average time between SEV-1/2 incidents. Higher = better.	≥ 30 days	~9 days (2026-08-29 + 2026-08-10..18 streak, but streak was 1 root cause = 1 incident)	Post-mortem index	Any SEV-1 within 7 days of a prior SEV-1
M4	**Deployment frequency**	Number of production deploy-ments per month. Lower = suspicious (no progress); higher = suspicious (no review).	≥ 1 per week for non-emergency; emergency hotfix separate	~3 per month (rough estimate)	git log + heartbeat tag	< 1 per month OR > 20 per month

#	Metric	Definition	Target	Today's baseline	Source	Control limit (alert when…)
M5	**Change failure rate**	% of deployments that cause a SEV-1/2 within 7 days of deploy.	< 10%	~33% (1 SEV-1 out of 3 deploys in August 2026, haritha_hospital install)	git log cross-ref LEARNINGS.md	> 25% over any 30-day window
M6	**Backup success rate**	% of cron slots that produce a `BACKUP_OK` sentinel line + offsite rsync artefact.	100% (every slot, every day)	~85% (post-LEARNINGS #113/#114 fix; pre-fix streak = 0%)	prod_backup.sh log + offsite listing	Any slot failure; OR no `BACKUP_OK` in last 26h
M7	**Login latency**	Time from login POST to first response, p95.	< 500 ms p95	not measured today	needs Frappe monitoring module	> 1000 ms p95 for 3 consecutive probes
M8	**DB query latency**	Slow-query log review: any query > 1s logged in 24h.	0 slow queries > 5s per day	not measured today (Frappe logs `tabError Log` but no automated review)	`tabError Log` + MariaDB slow-query log	> 5 slow queries > 1s in any 24h window

3a.2 Metric ownership and decision

Each metric has an owner and a decision it drives:

Metric	Owner	Decision it drives
M1 Availability	PA + VN	"Should we escalate to SEV-1?" / "Did the last change regress uptime?"
M2 MTTR	VN	"Is our incident response getting better?"
M3 MTBF	VN	"Are we regressing into more incidents?"
M4 Deployment frequency	PA	"Are we shipping improvements?"
M5 Change failure rate	VN	"Should we slow down deploys and review the process?"

Metric	Owner	Decision it drives
M6 Backup success rate	PA	“Is the backup pipeline broken?”
M7 Login latency	PA	“Is the system fast enough for daily use?”
M8 DB query latency	PA	“Are we shipping schema or query regressions?”

3a.3 What we measure TODAY vs what we DON’T

Metric	Measured today?	How	Gap
M1 Availability	**Partial** — heartbeat reports “site up/down” but no rolling 30-day uptime %	Heartbeat probe	Need 30-day rolling window calculator
M2 MTTR	**Partial** — post-mortems record timeline but not aggregated	Post-mortem timeline	Need aggregation across post-mortems
M3 MTBF	**Partial** — incidents live in `memory/YYYY-MM-DD.md` but no index	Memory log	Need incident index
M4 Deployment frequency	**NO** — could compute from git log	not computed	Easy fix: script
M5 Change failure rate	**NO** — could compute from git log + LEARNINGS.md	not computed	Easy fix: script
M6 Backup success rate	**YES** — BACKUP_OK sentinel + log-tail probe	05-operations-security §3.1	None — live
M7 Login latency	**NO**	not measured	Need Frappe Monitoring module + probe
M8 DB query latency	**NO**	not measured	Need slow-query log parsing

3a.4 Tooling

Today (v1):

- **Heartbeat** — daily subagent probes; logs to `memory/heartbeat-state.json`.
- **Git log** — every commit timestamped; commit messages cite LEARNINGS IDs.
- **`pberpprod_backup.sh`** — emits `BACKUP_OK` sentinel; heartbeat tails the log.
- **Post-mortem file** — timeline table per incident; this is the M2 source.
- **Memory log** — daily subagent logs; the raw data.

Tomorrow (v2):

- **Frappe Monitoring module** — built-in dashboards for request count, slow queries, error count. Currently disabled; enable and configure for prod.

- **Custom Daily Ops card** — a Frappe Desk page summarising M1-M8 once per day, replacing ad-hoc heartbeat parsing.
- **Simple CSV/JSON aggregator** — Python script that reads heartbeat log + post-mortem index + git log and emits the 8 metrics as a table.

Out of scope (v3+):

- Prometheus + Grafana. Powerful but operationally heavy for a single-operator setup. Revisit when second operator joins.
- ELK / Loki for log aggregation. Same reasoning.

3b. Concrete Examples (Haritha history)

Example 1 — M6 Backup success rate caught the silent streak (LEARNINGS #113, #114)

- **What the metric would have shown.** Between 2026-08-10 and 2026-08-18, `pberp-prod_backup.sh` ran 4×/day. The cron daemon reported "completed normally". Disk usage on backup volume grew (false reassurance). Offsite rsync was empty.
- **What the metric would have done.** M6 = (successful_slots / total_slots) over the window. From 2026-08-10 to 2026-08-18, M6 = 0%. A daily probe that compared offsite listing to local listing would have flagged it within 26h, not 8 days.
- **Why this is the canonical example.** LEARNINGS #113/#114 are the textbook case of "metric was 0% but nobody noticed because nobody looked". The lesson: a metric you don't read is not a metric.
- **Today.** 05-operations-security §3.1 + this metric definition together close the loop. The sentinel line is the source of truth.

Example 2 — M2 MTTR measured post-2026-08-29 outage

- **What the metric would have shown.** SEV-1 from 03:06 IST (detection) to 03:17 IST (all-clear) = 11 min. Plus 03:17-03:20 IST secondary fix (LEARNINGS #154 password drift) = 14 min total. M2 SEV-1 = 14 min.
- **Why this matters.** The metric is below the 30-min target. The 2026-08-29 case becomes a baseline: future SEV-1s should be ≤ 30 min. If we see MTTR creep to 25 min consistently, that's a signal we're getting slower — investigate before we cross the threshold.
- **Today.** The post-mortem timeline in 05.2 §"Part 2" is the source of truth; aggregation is manual until the v2 tooling lands.

Example 3 — M5 Change failure rate spikes on haritha_hospital install

- **What the metric would have shown.** August 2026 had 3 deploys:
  1. `haritha_hospital` skeleton push → no incident
  2. `haritha_hospital` fixtures export → no incident
  3. `haritha_hospital` install on both envs → SEV-1 (LEARNINGS #153)
- **M5 = 1/3 = 33%.** That's 3× the 10% target. Signal: app-install deploys need a different runbook path that includes `docker restart` (LEARNINGS #153 fix).
- **Today.** The metric would have flagged this. Without it, the install runbook looked fine until the outage proved otherwise.

Example 4 — M7 Login latency — gap example

- **What the metric would have shown.** Not measured today. Venkat's perception ("the system feels slow today") could be:

  - Network blip (not the system).
  - DB query regression (system issue, fix).
  - Browser cache (user issue, FAQ).
- **Without the metric:** all three look the same. The operator investigates one cause; finds nothing; moves on; problem recurs.
- **With the metric:** login latency = X ms, baseline = Y ms. If X » Y, the system is the cause. If X ≈ Y but user reports slowness, it's user-side.
- **Why this is in the "gap" section.** We don't measure it yet. Adding it is a v1 action (§9 immediate).

Example 5 — M8 DB query latency — the silent query regression

- **What the metric would have shown.** A schema or migration change introduces a slow query (e.g., `SELECT * FROM tabX WHERE y > 0` without an index). For low traffic, the slow query is invisible. For high traffic, it tanks the system.
- **Today.** No automated review of `tabError Log` or MariaDB slow-query log. We'd find out when a user complained.
- **With the metric:** daily review of "queries > 1s in last 24h". Regression caught before user impact.

Example 6 — M3 MTBF and the over-counting problem

A 6-day silent-failure streak (LEARNINGS #113/#114) is ONE incident (one root cause), not 24 (one per failed cron slot). If we counted failed cron slots as incidents, MTBF would artificially collapse and we'd chase the wrong signal. - **Today.** No incident index; risk of over-counting. - **Decision rule.** MTBF counts incidents by root cause, not by symptom. Failed cron slots are SYMPTOMS of the backup-script incident. The heartbeat probe that fires "no BACKUP_OK in 26h" is the single incident detector.

Example 7 — M4 Deployment frequency — the under-counting problem

If we count only `git push` to `main` as deploys, we under-count (we deploy via containers that may not change git state). If we count every docker pull, we over-count (cached layers don't deploy anything new). - **Decision rule.** Deployment = change to `apps/` OR `compose.yaml` OR `sites/assets` committed to `main`. Countable from git log: `git log --since="..." --name-only --pretty=format:"%h" main`.

4. Responsibilities

Role	Responsibilities
Venkat Narasimha (Owner)	Approves the metrics list. Sets annual targets. Reviews monthly QPM summary. Owns the control-limit escalations.
Processbricks admin	Implements measurement (heartbeat probes, scripts). Maintains the post-mortem timeline aggregation. Surfaces anomalies via heartbeat alerts.
Subagents (automation)	Run probes on schedule. Do not interpret metrics ("is this an incident?") — that decision goes to the operator.
Future operators	Read the metrics catalogue first. Don't ask "is the system OK?" — read M1, M6, M7.

5. Compliance Measurement

Check	Frequency	Owner	Source of truth
All 8 metrics have a current value (not “unknown”)	Monthly	PA	QPM dashboard / aggregation
All 8 metrics have a target	Annual	VN	this doc §3a.1
All 8 metrics have a control limit	Annual	VN	this doc §3a.1
All 8 metrics are read at least once per month	Monthly	PA	review log
Anomalies within control limits are documented (even if not alerted)	Monthly	PA	anomaly log
Quarterly review of metric usefulness (drop unused metrics)	Quarterly	VN	review notes
Annual target reset	Annually	VN	this doc §3a.1 updated
Annual aggregation of M2/M3 from post-mortem index	Annually	PA	post-mortem index script

KPI dashboard (informal):

KPI	Target	Source
% of metrics with current value	100%	QPM dashboard
% of metrics read monthly	100%	review log
Mean time from “metric broken” to “metric fixed”	≤ 14 days	QPM dashboard gaps
M2 MTTR SEV-1	≤ 30 min	post-mortem timelines
M3 MTBF (rolling 30 days)	≥ 30 days	post-mortem index
M5 Change failure rate	< 10%	git log cross-ref

6. Exceptions

1. **No Prometheus / Grafana in v1.** Acknowledged limit; revisit when second operator joins.
2. **M4, M5, M7, M8 not measured today** — explicit gaps (§3a.3). Until they’re measured, they are aspirational targets, not enforced.
3. **Annual reset of targets** is allowed (e.g., if M6 was 100% all year, we may raise the bar to “BACKUP_OK within 30 min of slot”). Document the change in §3a.1.
4. **All other exceptions** follow 01-info-security §6.

6a. Edge Cases & Decision Matrix

Edge case 1 — A metric breaches its control limit

- **Trigger.** M1 prod availability drops to 99.4% for a 7-day window (control limit: < 99.5% prod for any 7-day window).
- **Decision matrix.**

Action	Allowed?	Why
Page Venkat immediately	YES	Control limit breach = signal, not noise
Spawn diagnostic subagent	YES	Same pattern as 07 §3.4
Auto-mitigate (e.g., restart)	NO	Metric doesn't tell us WHY; investigation comes first
Update the control limit	NO during the breach; YES after post-mortem	Avoid gaming the metric

- **Default action.** Page + diagnostic subagent + investigation. Post-mortem decides if the limit was wrong.

Edge case 2 — A metric is consistently above target (better than expected)

- **Trigger.** M7 login latency p95 = 200ms for 30 days running; target = 500ms.
- **Decision matrix.** Options:
  - Raise the bar (target = 300ms; control limit = 600ms).
  - Document as "headroom available" but don't change targets.
  - Celebrate (yes, this is allowed).
- **Default action.** Document as headroom. Don't auto-raise targets; that's an annual review decision.

Edge case 3 — A metric can't be measured (data source broken)

- **Trigger.** Heartbeat probe for M1 fails (network down). M1 has no current value.
- **Decision matrix.**

Action	Why
Mark M1 as "unknown" in the dashboard	HONEST — better than fabricating
Page Venkat	YES — heartbeat down is itself an incident
Don't restart the probe mid-incident	Could mask root cause

- **Default action.** Mark unknown; page. Document the gap in the post-mortem if the outage extends.

Edge case 4 — Two metrics conflict

- **Trigger.** M4 deployment frequency = 20/month (way above target). M5 change failure rate = 30% (also above target).
- **Decision matrix.** Could mean (a) we're shipping too fast for review, or (b) we have a quality problem in review. The metrics say "something is wrong" but not which.
- **Default action.** Spawn a diagnostic subagent for "deployment review process". Likely a PDCA cycle on the deploy runbook (04.1).

Edge case 5 — A new metric is proposed

- **Trigger.** Admin wants to track "average tenant log size" — never been measured.
- **Decision matrix.** PDCA cycle. Plan = "define metric + target + source". Do = "implement measurement". Check = "review 30 days of data". Act = "standardise in §3a.1, or abandon if metric is uninformative".
- **Default action.** Open a PDCA cycle per 10-process-improvement. The metric only becomes "official" after the cycle's Act step.

Edge case 6 — A metric is no longer useful

- **Trigger.** Quarterly review: nobody reads M8 (DB query latency); auto-review catches nothing actionable.
- **Decision matrix.** Two paths:
  - Improve the metric (e.g., add a real slow-query threshold).
  - Remove the metric.
- **Default action.** Try to improve first. Remove in the annual review if still unused. Document the removal.

Edge case 7 — A metric breaches on a single bad sample

- **Trigger.** M7 login latency = 1500ms for one probe; other 29 probes in the month were 200-400ms.
- **Decision matrix.** Single-sample breach ≠ signal. Apply Western Electric / Nelson rules: alert only if 1 point > 3σ, OR 2 of 3 > 2σ same side, OR 8 consecutive same side, OR 6 trending.
- **Default action.** For v1: alert on single-sample if it's > 2× control limit; otherwise log and watch.

Edge case 8 — A metric's source is the operator

- **Trigger.** M2 MTTR is computed from post-mortem timelines. If Venkat writes a sloppy post-mortem, M2 is wrong.
- **Decision matrix.** The metric is only as good as the data source. Strengthen the source (05.2 template enforces timeline format).
- **Default action.** Audit the post-mortem template quarterly; flag missing fields.

7. Related Documents

- 09-cmm-maturity-assessment.md — HH-CMM-01 — QPM feeds the maturity score.
- 10-process-improvement.md — HH-CMM-02 — PDCA Check steps use QPM metrics.
- 12-defect-prevention.md — HH-CMM-04 — Defect prevention's metrics come from QPM.
- 07-incident-management.md — Incidents generate M2 (MTTR) and M3 (MTBF).
- 08-business-continuity.md — DR drills generate M6 (backup) data.
- 05-process/05.2-post-mortem.md — Source of M2 + M3 raw data.
- 04-runbooks/04.2-daily-ops.md — Daily heartbeat, the v1 measurement engine.
- ../../../../.learnings/LEARNINGS.md — Source of M5 (change failure rate) cross-reference.
- ../../../../MEMORY.md — Tech stack + container names.

8. Revision History

Version	Date	Author	Changes
1.0	2026-08-29	venkat-narasimha	Initial

9. Implementation Checklist

Concrete actions derived from this policy. Owner initials: VN = Venkat Narasimha; PA = Processbricks admin. Status as of 2026-08-29.

Immediate (this week)

- ☐ **Enable Frappe Monitoring module on prod** (Settings → Monitoring → enable). Owner: PA. Target: 2026-09-05. Status: Not Started.
- ☐ **Author the v1 metrics aggregation script** (`scripts/qpm_aggregate.py`) that computes M1-M6 from existing data. Owner: PA. Target: 2026-09-05. Status: Not Started.
- ☐ **Wire the aggregation script into the daily heartbeat** so metrics show up in the daily summary. Owner: PA. Target: 2026-09-05. Status: Not Started.

Short-term (2026-Q3)

- ☐ **Compute baselines** for M1-M6 from historical data (heartbeat logs, post-mortems, git log). Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **Implement M7 login latency probe** (curl from heartbeat subagent to `/login` with timing). Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **Implement M8 DB query latency review** (daily cron parses `tabError Log` for queries > 1s). Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **First monthly QPM review** with Venkat. Owner: VN. Target: 2026-09-30. Status: Not Started.
- ☐ **Author the QPM dashboard** at `desk#monitoring/qpm-overview` (or markdown file if Desk module unavailable). Owner: PA. Target: 2026-09-30. Status: Not Started.

Medium-term (2026-Q4)

- ☐ **All 8 metrics measured and reported** weekly. Owner: PA. Target: 2026-12-31. Status: Not Started.
- ☐ **First control-limit breach** handled via the §6a Edge 1 decision matrix. Owner: PA. Target: 2026-12-31. Status: Not Started.
- ☐ **Quarterly metric usefulness review** (drop unused, improve weak). Owner: VN. Target: 2026-12-31. Status: Not Started.
- ☐ **Annual target reset** (this doc §3a.1 updated for 2027). Owner: VN. Target: 2027-01-15. Status: Not Started.

Long-term (2027+)

- ☐ **Prometheus + Grafana** for time-series visualisation. Owner: VN. Target: TBD. Status: Not Started.
- ☐ **Anomaly detection** (Western Electric / Nelson rules automated). Owner: VN. Target: TBD. Status: Not Started.
- ☐ **Predictive metrics** (e.g., disk-full-in-N-days forecast). Owner: VN. Target: TBD. Status: Not Started.

Recurring verification (runs forever)

- ☐ **Daily heartbeat probe** — runs all 8 metrics where measurable. Owner: PA. Frequency: daily. Status: Done (M6); partial (others).
- ☐ **Weekly summary** — table of all 8 metrics in daily heartbeat. Owner: PA. Frequency: weekly. Status: Not Started.
- ☐ **Monthly review** — Venkat reads the summary, decides actions. Owner: VN. Frequency: monthly. Status: Not Started.
- ☐ **Quarterly metric audit** — drop unused, improve weak. Owner: VN. Frequency: quarterly. Status: Not Started.
- ☐ **Annual target reset** — update §3a.1 with new targets based on year. Owner: VN. Frequency: annually. Status: Done (this revision).

Measure what matters. Measure it honestly. Read the metric before deciding — and if nobody's reading it, remove it. Document or repeat.